



The F1R3Games Portal: Design Document

Single sign-on with a built-in wallet, galleries of game plays, launching games, and invitations, on a F1R3Node-Rust shard

Version 0.1 (draft for review)

Lucius Gregory Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

5 October 2026

Abstract

This document designs the F1R3Games Portal: the single web entry point to the F1R3Games suite (F1R3Pix, F1R3Beat, F1R3Ink, F1R3SideChat, F1R3Skein, and the games that follow them). A person arrives with nothing but a browser, creates or imports a secp256k1 private key, and from then on is signed on to every game. A built-in wallet holds the key, pays for every deploy the person makes, shows balances and history, and moves funds. The person can browse the gallery of past plays of every game (including, for F1R3Skein, the separate galleries of tunes and of performances), launch a new instance of any game, and bring other people into it by sharing her contacts with the Portal and sending invitations.

The backend is a F1R3Node-Rust shard, reached through the Embers service pattern. The design is written to *conform*: to F1R3Node-Rust first (its HTTP and gRPC interface, its system contracts and their current names, its node roles and its proposing policy), then to Embers (its service layering, its prepare/sign/send protocol, its typed rendering of Rholang, its wallet API and client SDK), then to F1R3Sky (its wallet file format and its wallet flows). The earlier F1R3Games work, the portal prototype, the per-game Shard/services and the contact dialogues, is treated as research: its concepts are carried forward, its code is not. The design is grounded in a review of F1R3Node-Rust master at 95be0d4 and feature/cost-accounted-rho at 068d51c, Embers at 28e50d4, F1R3Sky at 57dcae5, F1R3Games main at ed61943 and v2 at 18a675a, F1R3Gaze at f6ee26a, and ign1t10n at 46faa56.

Contents

1	Scope and conventions	4
1.1	What the Portal delivers	4
1.2	Non-goals for this version	4
1.3	Order of authority	4
1.4	Conventions	5
2	Requirements	5

3 Findings	6
3.1 What the Portal conforms to	6
3.2 What the earlier F1R3Games work teaches	7
3.3 How conformance closes the open questions	8
4 Principles	8
5 Architecture	9
5.1 Components	9
5.2 Why an Embers domain and not a new service	10
6 Identity and sign-on	10
6.1 The key and the address	10
6.2 Creating, importing and exporting	11
6.3 The keystore	11
6.4 What “signed on” means	11
6.5 Profiles	11
7 The wallet	11
7.1 Functions	11
7.2 Signing rules	12
7.3 Allowances	12
8 The games environment on the shard	13
8.1 Shape	13
8.2 Domains and methods	13
8.3 Instances	14
8.4 Play records	14
8.5 Ordering and paging	14
8.6 Rendering and injection	14
9 Games	15
9.1 The game manifest	15
9.2 The host protocol	15
9.3 Launch	16
9.4 The games	16
9.5 F1R3Skein on Vision Pro	17
10 Galleries	17
10.1 What the person sees	17
10.2 No sign-on to read	17
10.3 Trust in the view	17
11 Contacts and invitations	18
11.1 Sharing contacts with the Portal	18
11.2 The invitation	18
11.3 Delivery	18
11.4 Redemption	19
11.5 Paying for a newcomer’s first deploy	19

12 Engagement	19
13 Security and privacy	20
14 Deployment	20
15 The F1R3Gaze rendition	21
16 Testing and acceptance	21
17 Work packages and schedule	21
18 Obligations and decisions	22
18.1 Obligations	22
18.2 Open decisions	22
A Sketch of the invitation methods	23
B Glossary	24

1 Scope and conventions

1.1 What the Portal delivers

After this design is implemented, a person with an ordinary web browser can:

1. **Sign on with a private key.** Create a new secp256k1 key, or import one saved by F1R3Sky or F1R3Gaze, and use it as the sole credential for every game. There are no passwords held by any server and no email-based accounts.
2. **Use a built-in wallet.** See the balance and history of the address the key controls, receive funds, send funds, see what each game has cost, and approve or refuse every charge before it is made.
3. **Browse previous games.** Scroll the gallery of plays of any game, or a feed across all games, without launching anything and without signing on.
4. **Launch a game.** Start a new instance of any game, which appears on the shard as an object the person hosts, and open the game on it.
5. **Invite others.** Share contacts with the Portal (from a file, a platform's contact picker, or by hand), choose whom to invite, and deliver invitations through the person's own channels. An invitation, when redeemed, brings the invitee into the instance, creating a key for them if they have none.

1.2 Non-goals for this version

- The internal mechanics of any game. Each game owns its rules, its on-chain state and its user interface; the Portal owns identity, payment, instances, galleries, invitations and the contract between itself and the games (§9).
- Custodial accounts or key recovery by the operator. The key is the person's, as it is in F1R3Sky and F1R3Gaze.
- Fitness and selection for the evolutionary games. The Portal records engagement (§12); how F1R3Skein and F1R3Beat turn engagement into reproduction is theirs to decide.
- Delivery of invitations by an operator-run mail server or bot. The design leaves room for it (§11.3) and does not depend on it.
- The f1r3lang rendition of the Portal for F1R3Gaze. It is sketched as a later phase (§15).

1.3 Order of authority

Where sources disagree, this design resolves the disagreement in a fixed order:

1. **F1R3Node-Rust** master: the node's API reference (docs/node), its system processes (rho-runtime.rs in the interpreter), its system contracts (casper/src/main/resources) and its node roles. Features present only on feature/cost-accounted-rho are designed *for*, never depended *on*, until they reach master.
2. **Embers**: the service layering in packages/embers, the prepare/sign/send protocol in api/common.rs, the typed Rholang rendering in packages/firefly-client, the environment-contract pattern in templates/common/insert_signed.rho, the wallet API, and the client SDK @f1r3fly-io/embers-client-sdk.

3. **F1R3Sky**: the wallet key file, the create-and-download and upload-to-add wallet flows, and the use of the Embers SDK from a React client.
4. **F1R3Gaze and ign1t10n**, where they have settled a question the three above leave open (the signing-safety rules of F1R3Gaze’s wallet; ign1t10n’s local shard topology).
5. **The earlier F1R3Games work**, as research only.

1.4 Conventions

MUST, MUST NOT, SHOULD and MAY are used in the sense of RFC 2119. “Address” means a F1R3Cap address as the Embers SDK derives it from a secp256k1 public key. “Deploy” means a signed DeployData as F1R3Node-Rust accepts it at POST /api/deploy. “Validator” and “observer” mean a bonded node and a read-only node respectively. “Instance” means one play of one game: what a player launches and invites others into. “Play record” means the gallery entry an instance, or a part of one, leaves behind.

2 Requirements

Requirement 1 (Single entry point). All games MUST be reachable from one Portal, and a person signed on to the Portal MUST be signed on to every game launched from it without a further sign-on step.

Requirement 2 (Key-based sign-on). The only credential MUST be a secp256k1 private key. The Portal MUST create keys and import keys in the F1R3Sky wallet file format, and MUST export keys in that format, so that one key works in the Portal, F1R3Sky and F1R3Gaze.

Requirement 3 (Built-in wallet). The Portal MUST contain a wallet that shows balance and history, receives and sends funds, and signs and pays for every deploy the Portal or a game makes on the person’s behalf, after the person’s consent or within an allowance the person has set.

Requirement 4 (Galleries). Every game MUST have a gallery of plays, browsable from the Portal without launching the game and without signing on. F1R3Skein MUST have two galleries, tunes and performances, cross-linked.

Requirement 5 (Launch). A signed-on person MUST be able to start a new instance of any game, and the instance MUST exist on the shard before any invitation to it is issued.

Requirement 6 (Contacts and invitations). A person MUST be able to share contacts with the Portal and invite them to an instance or to the platform. One contact dialogue MUST serve every game, except that the F1R3Skein browser does not use it.

Requirement 7 (The shard is the backend). Every durable fact about players, instances, plays, invitations and engagement MUST live on the F1R3Node-Rust shard. Any service in front of the shard MUST be stateless in the sense that it can be destroyed and rebuilt from the shard and its own configuration.

Requirement 8 (Open to new games). Adding a game MUST require registering it, not changing the Portal.

Requirement 9 (Conformance). The Portal MUST use F1R3Node-Rust’s interfaces and system names as they are on master, Embers’s protocol and libraries, and F1R3Sky’s wallet format, and MUST NOT introduce a parallel mechanism where one of these already provides one.

3 Findings

These are the facts the design rests on. The first group fixes what the Portal must conform to; the second records what the earlier F1R3Games work taught and what it got wrong, so that the mistakes are not repeated.

3.1 What the Portal conforms to

Finding 1 (Deploys). F1R3Node-Rust accepts deploys at `POST /api/deploy` on validators only. The body is data (term, timestamp, phloPrice, phloLimit, validAfterBlockNumber, shardId, and an optional expirationTimestamp), deployer (hex public key), signature and sigAlgorithm "secp256k1". The response is the deploy id. Deploy status is at `GET /api/deploy/{id}` and `GET /api/deploy-finalization-status/{sig}` (docs/node/api-reference.md).

Finding 2 (Reads need an observer). `explore-deploy`, `estimate-cost`, `balance`, `registry` and the validator queries run only on read-only nodes; a validator answers 400 `readonly_node_required`. `GET /api/status` reports `isValidator` and `isReadOnly`. An exploratory deploy returns what is sent on the *first* unforgeable name of its new, which must carry no URI.

Finding 3 (Names can be known before a deploy). `POST /api/prepare-deploy` with a deployer, a timestamp and a count returns the unforgeable names that deploy will create, and the validator's next sequence number for `validAfterBlockNumber`. A client can therefore know an object's identity on the shard before the deploy that creates it is accepted.

Finding 4 (Events). `/ws/events` streams `block-created`, `block-added`, `block-finalised` (each with the deploys it contains: id, cost, deployer, errored) and, on observers only, `transfers-available`.

Finding 5 (Proposing). `Propose` is an admin operation (`POST /api/propose` on port 40405; gRPC `ProposeService` on 40402). Validators propose on their own when heartbeat proposing is enabled (`-heartbeat-enabled`). Embers calls `propose` once, at bootstrap, after deploying its environments (packages/embers/src/main.rs).

Finding 6 (System names on master). The runtime registers `rho:vault:address`, `rho:system:deployerId:ops`, `rho:deploy:data`, `rho:block:data`, `rho:registry:ops`, `rho:crypto:blake2b256Hash`, `rho:crypto:secp256k1Verify`, `rho:io:devNull` and `rho:execution:abort`; system contracts include `rho:vault:system` (`SystemVault:findOrCreate`, `deployerAuthKey`, `unforgeableAuthKey`, `transfer`), `rho:vault:multiSig`, `rho:lang:treeHashMap`, `rho:registry:insertSigned:secp256k1` and `rho:registry:lookup`. The `rho:rev:*` and `rho:rchain:*` names used by older code are gone.

Finding 7 (Capabilities and offered funding are not on master). The capability registry `rho:system:capabilities` (bounded-use handles) and the offered-funded deploys with private purses (`vault_payer.rs`, `SystemVault.transferBatch`, settlement receipts) exist on `feature/cost-accounted-rho`; the latter's own proposal records that final qualification is pending. Neither is on master.

Finding 8 (Embers is the service pattern). Embers is a stateless Rust service (poem-openapi) laid out as `api/<domain>`, `domain/<domain>`, `blockchain/<domain>`, `templates/<domain>/*.rho`. Each domain deploys a versioned environment through `rho:registry:insertSigned:secp256k1` under its own environment key, re-initialising only when the on-chain version is lower; state lives

in `treeHashMaps`; the caller's address is derived inside the contract from `rho:deploy:data` and the vault-address from `fromDeployerId` operation. Writes use a two-step protocol: `/prepare` returns the deploy bytes the server rendered plus a token (a JWT binding a hash of the request and response, valid one day); the client signs the bytes and posts them with the token to `/send`. The key never reaches the server. Templates are rendered by `firefly-client`, whose typed `Value` (tuple, list, set, map, nil, bool, int, string, bytes, URI) renders literals; user strings never reach a template as raw text.

Finding 9 (Embers already has the wallet and versioned records). The wallets domain serves `GET /:address/state`, `POST /transfer/prepare|send`, `POST /boost/prepare|send` and `GET /:address/deploy`s. The agents and OSLF domains store per-address records with versions and a "latest" alias, and project headers in list views by omitting the heavy field. The testnet domain funds test wallets (`POST /testnet/wallet`).

Finding 10 (F1R3Sky's wallet). F1R3Sky creates a wallet with the SDK's `PrivateKey.new()` and immediately saves it as `<address>.json` (`serializeKey`); it adds a wallet by uploading that file (`deserializeKey`); it keeps wallets only in memory, so a reload requires the file again; each wallet carries an `EmbersApiSdk` bound to its key. The file is `{"keyType":"secp256k1","value":"<64 hex>","valueFormat":"hex"}`, as F1R3Gaze also documents and tests.

Finding 11 (F1R3Gaze's signing-safety rules). F1R3Gaze's wallet decodes prepared bytes strictly, checks shard id and that `phloPrice × phloLimit` is within a fee cap, requires a transfer term to be exactly Embers's template with the user's own values, and refuses any program that binds the deployer's identity, from which a program could obtain the vault's authority and spend.

Finding 12 (Name drift in Embers). Embers's templates (last commit February 2026) bind `rho:rev:address`, and F1R3Gaze's allow-list names `rho:rev:address` and `rho:rchain:deployId`; master has `rho:vault:address` and `rho:system:deployId`. By the order of authority, the Portal's templates use the master names, and the Embers templates the Portal relies on must be brought to them (Obligation 1).

3.2 What the earlier F1R3Games work teaches

Finding 13 (Concepts worth keeping). The prototype established the shape of the experience: a landing page, identity creation and import, a dashboard of game cards each offering *launch* and *gallery*; a sigil avatar derived from the address; a contact-import step with sources (email, CSV, platform handles, manual) feeding an invite carousel and a lobby; F1R3Skein's distinction between tunes and performances; SideChat's free reader tier. All of these are kept.

Finding 14 (Mistakes not to repeat). 1. Identity used ECDSA P-256 keys and the artifact storage shim, not secp256k1 and a shard.

2. Accounts, contacts, sessions and invites lived on public string-named channels (`@f1r3-accounts` and so on), consumed and re-sent whole: any deployer could overwrite them, and every writer contended for one datum.
3. Contacts, which are other people's personal data, were written to the chain in plaintext.
4. Rholang was built by string interpolation of user input.
5. No game had a notion of an instance; each kept one global state.
6. Clients called `propose`.

7. Shared code was shared by copying (`key-management.ts`, with a Python slice in TypeScript, duplicated across games).

3.3 How conformance closes the open questions

The review of 5 October 2026 left several questions open. Taking F1R3Node-Rust, Embers and F1R3Sky as the authorities closes them:

Resolution 1 (Key type and format). `secp256k1`, F1R3Cap addresses, F1R3Sky's key file, all through the Embers SDK (Findings 1, 10).

Resolution 2 (Where the backend logic lives). In a Embers domain, games, built exactly as Embers's other domains are (Finding 8); the Portal speaks to Embers, and Embers speaks to the shard. No bespoke service, no direct node access from the browser for writes.

Resolution 3 (Cross-origin access to the node). Moot for writes and most reads: the browser talks to Embers, whose own CORS policy governs. The `ign1t10n` decision that the node refuses foreign origins stands.

Resolution 4 (Signing in the browser). With the Embers SDK, as F1R3Sky does. The F1R3Gaze crates are not needed in the browser; F1R3Gaze remains compatible because the key file and address derivation are the SDK's.

Resolution 5 (Invitations). `rho:system:capabilities` is not on master, so invitations are a contract in the games environment built only from master primitives (§11), with a handle shaped so that it can later be backed by a capability registration without changing the link format.

Resolution 6 (Proposing). Nobody in the Portal path proposes. Deployments run validators with heartbeat proposing (Finding 5); Embers's bootstrap propose is the only exception, as it is today.

Resolution 7 (The Skein instrument on Vision Pro). It imports the same key file (F1R3Sky format) and talks to the same Embers domain; no delegation mechanism is invented for it.

Resolution 8 (Allowances and sponsored onboarding). On master, an allowance is enforced by the wallet in the client and onboarding is funded by an ordinary transfer (or Embers's testnet faucet). Private purses from offered funding replace both when they reach master (§7.3).

4 Principles

Principle 1 (The key never leaves the person's device). Private keys are created, held and used only in the person's browser (or F1R3Gaze, or the Vision Pro app). Embers prepares; the client signs; Embers sends.

Principle 2 (The shard is the database). Embers and any indexer hold no state that cannot be rebuilt from the shard. Losing every server loses no player's data.

Principle 3 (Sign only what can be read). The wallet signs a prepared deploy only after decoding it and recognising its term as a registered template filled with values the person supplied or approved.

Principle 4 (Contacts belong to the person). Contact data stays on the person's device, encrypted at rest. Nothing about a contact is written to the shard in plaintext. Only an invitation's commitment goes on chain.

Principle 5 (Authority comes from the deployer, inside the contract). Every write method derives the caller’s address from `rho:deploy:data` and `rho:vault:address`, as Embers does, and authorises against it. No method trusts an address passed as an argument.

Principle 6 (Games plug in; the Portal does not know their rules). A game is a registered manifest, its own templates and its own user interface. The Portal provides identity, payment, instances, galleries, invitations and engagement through one protocol.

Principle 7 (Reads at observers, writes at validators, propose by heartbeat). As F1R3Node-Rust prescribes (Findings 1, 2, 5).

5 Architecture

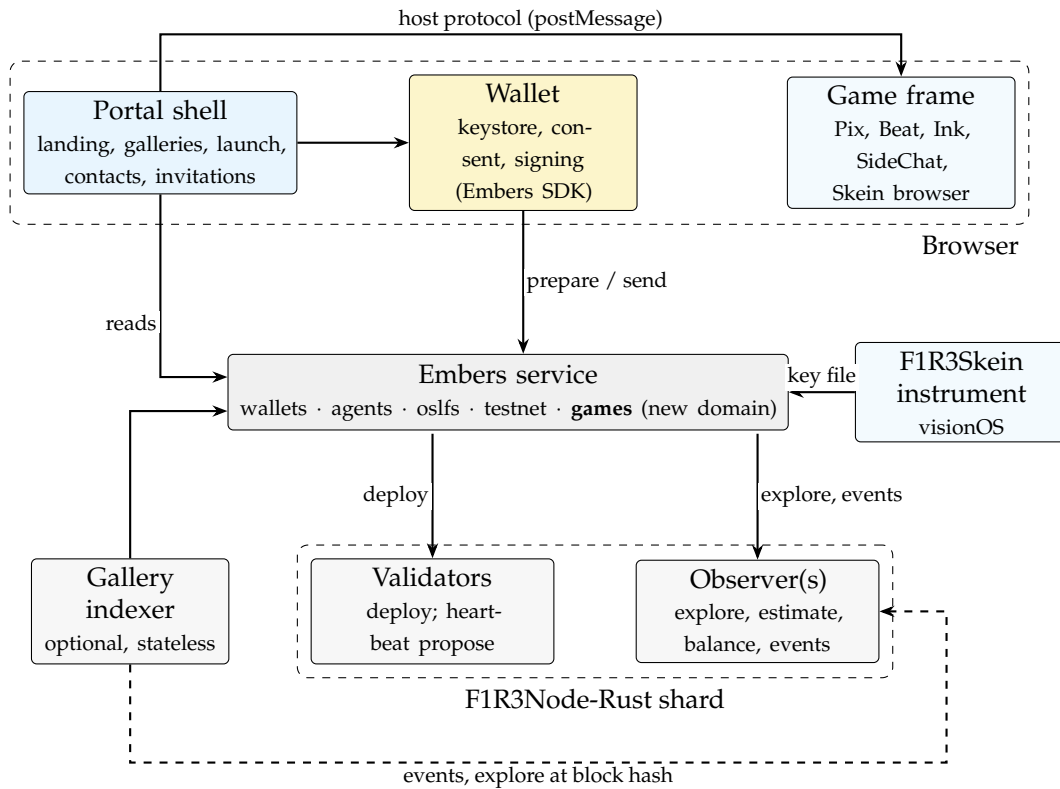


Figure 1: Components. The browser never holds a server session and never talks to a validator; it talks to Embers, which renders, deploys and reads. The shard holds the games environment, the games’ own environments and SystemVault. Solid arrows are requests.

5.1 Components

Component	Responsibility
Portal shell	A React web application (as F1R3Sky is), served as static files. Landing, sign-on, dashboard of games, galleries, launch, the contact dialogue, invitation redemption, and the host side of the game protocol.
Wallet	A module of the shell, isolated behind a narrow interface. Holds the encrypted keystore, decides consent, signs with the Embers SDK, and calls Embers's wallets domain for balance, history and transfers.
Game frame	Each web game runs in a sandboxed frame on its own origin and reaches identity, payment and the shard only through the host protocol (§9.2).
Embers games domain	New. Endpoints and templates for profiles, the game registry, instances, play records, invitations and engagement, built as Embers's other domains are; deploys and maintains the games environment.
Embers (existing)	Wallets (balance, history, transfers), testnet faucet, and the node clients and event subscriptions every domain shares.
Gallery indexer	Optional. Rebuilds ordered gallery views from <code>/ws/events</code> and exploratory reads, for scale; everything it returns carries the block hash it read at, so a client can check it against an observer.
F1R3Node-Rust shard	Validators accept deploys and propose by heartbeat; observers serve every read and the event stream.

5.2 Why an Embers domain and not a new service

The design of the earlier F1R3Skein development plan proposed a new `f1r3games-service` “modelled on Embers”. Conformance argues for going the whole way: a domain inside Embers inherits, without reimplementing, the node clients and their configuration, the environment-key handling, the prepare/send token, typed rendering, the event subscriptions, the OpenAPI schema from which the client SDK is generated, and the deployment. The Portal then uses one SDK for its wallet and its games, as F1R3Sky uses one SDK for its wallet and agents. Whether the domain lives in the Embers repository or in a crate that Embers links is Open decision 1; the protocol is the same either way.

6 Identity and sign-on

6.1 The key and the address

A person's identity is one `secp256k1` key pair. The public key, hex encoded uncompressed, is the deployer of every deploy the person makes; the address is the F1R3Cap address the Embers SDK derives from it, which is also the address of the person's vault in `SystemVault`. The Portal MUST obtain both from the SDK (`PrivateKey`, `getPublicKey()`, `getAddress()`) and MUST NOT reimplement the derivation.

6.2 Creating, importing and exporting

Create. `PrivateKey.new()`; then, before the person can continue, the Portal offers the key file for download exactly as F1R3Sky does (`<address>.json, serializeKey`). The Portal does not let a person proceed past creation without either downloading the file or explicitly acknowledging that losing the device loses the key.

Import. Upload of a F1R3Sky key file (`deserializeKey`), or paste of the 64-hex-digit value.

Export. The same file, on request, after re-unlocking.

A key created in the Portal **MUST** open in F1R3Sky and in F1R3Gaze, and vice versa; this is an acceptance test (§16).

6.3 The keystore

F1R3Sky keeps keys only in memory. The Portal must keep the person signed on across visits, so it keeps an encrypted keystore in the browser's IndexedDB:

- Each entry is the F1R3Sky key file encrypted with AES-256-GCM under a wrapping key, with the address and a label in clear.
- The wrapping key is derived either from a passphrase (PBKDF2-SHA-256 through WebCrypto, with a stored salt and a calibrated iteration count) or from a passkey using the WebAuthn PRF extension where the browser supports it (Open decision 2).
- Unlocked, the decrypted key lives only in the wallet module's memory, for a configurable idle period, after which the wallet locks.
- Several keys may be held; one is *active* and pays. This is F1R3Gaze's model and lets a person keep separate identities.

6.4 What “signed on” means

There is no login server. Embers is stateless and authenticates every write by the signature on the deploy it prepared; reads are public. Being signed on is therefore a client state: an unlocked active key, and the person's profile read from the games environment. Single sign-on across games follows from the games never holding keys: a game asks the shell, and the shell's wallet signs (§9.2). Nothing has to be shared between game origins except the protocol.

6.5 Profiles

A profile is a small record in the games environment keyed by address: display name, a sigil seed (the avatar is computed from the address, as in the prototype, so the seed is only a variation), optional tags, and creation and last-seen times. It is written by `profiles/save` and only the owner's deploy can write it. A person with no profile is still a valid player, shown by address and sigil.

7 The wallet

7.1 Functions

Function	Implementation
Balance and history	Embers GET <code>/wallets/:address/state</code> ; live updates from GET <code>/wallets/:address/deploys</code> .
Receive	Show the address and a QR code.
Send	Embers <code>/wallets/transfer/prepare</code> , verify, sign, <code>/send</code> .
Game spending	Per game and per instance, the phlo costs of deploys the person made there, read from the deploy records.
Consent	Every signature, except within an allowance, is preceded by a prompt naming the game, the action, the paying address and its balance, and the quoted cost.
Allowances	A per-instance budget within which a game's registered templates are signed without prompting (§7.3).
Keys	Create, import, export, label, choose active, remove.

7.2 Signing rules

The wallet adopts F1R3Gaze's rules (Finding 11), restated against Embers's protocol:

1. **Decode strictly.** The prepared bytes **MUST** decode as one `DeployData` with each field once, canonically encoded; the `shardId` **MUST** equal the configured shard.
2. **Cap the fee.** $\text{phloPrice} \times \text{phloLimit}$ **MUST** be within the person's fee cap, and within the remaining allowance if signing without a prompt.
3. **Recognise the term.** The term **MUST** equal a registered template (the Portal's own, or one listed by hash in the game's manifest) rendered with the values the request named. The wallet renders the template locally from the same typed values and compares; any difference refuses.
4. **No deployer authority to games.** A game template **MUST NOT** bind `rho:system:deployerId`, `rho:system:deployerId:ops` or `rho:vault:system` directly; these are reachable only through the games environment's methods, whose code is fixed and audited. Only the wallet's own transfer template touches the vault.
5. **Quote the cost.** Before prompting, the cost is estimated by Embers through `POST /api/estimate-cost` on an observer, passing the deployer key so identity-dependent terms are quoted correctly (`docs/node/api-reference.md`).

Rule 3 requires the client and Embers to render identically. The games domain **MUST** publish each template and its renderer's literal rules, and the wallet **MUST** implement `firefly-client`'s Value rendering in TypeScript, tested against vectors produced by the Rust renderer (Obligation 2).

7.3 Allowances

Games such as F1R3Pix make many small moves; a prompt per move is unusable. On launching or joining an instance, the wallet offers an allowance: a phlo budget, an expiry, and the set of template hashes the game's manifest declares for play. Within it the wallet signs without prompting, still applying every rule of §7.2, and shows a running total. On master the allowance is enforced only

by the wallet; the person’s whole balance remains at risk only from the wallet’s own code, which is the same trust F1R3Sky asks for.

When offered-funded deploys reach master (Finding 7), an allowance becomes a private purse funded by `SystemVault.transferBatch` and named in each move’s signed funding intent, so the budget is enforced by the shard and the person’s main balance is not exposed to a game at all. The protocol to games does not change.

8 The games environment on the shard

8.1 Shape

The games domain deploys one environment, in Embers’s pattern: a registry entry inserted with `rho:registry:insertSigned:secp256k1` under the `games_env_key`, holding `(version, bundle+{*games})`, re-initialised only when the deployed version is lower. Within it, one dispatch contract `games(@method, ...)` and state in `treeHashMaps`. Every write method begins by deriving the caller’s address:

```
new deployData(`rho:deploy:data`), vaultAddress(`rho:vault:address`),
  deployDataCh, callerCh in {
  deployData!(*deployDataCh) |
  for (_, @deployerId, _ <- deployDataCh) {
    vaultAddress!("fromDeployerId", deployerId, *callerCh)
  } |
  for (@caller <- callerCh) { /* authorise against caller */ }
}
```

8.2 Domains and methods

Domain	Methods	Authorisation
profiles	save, get, list	owner writes; anyone reads
registry	register, update, retire, get, list	operator key (later, governance); anyone reads
instances	create, join, leave, setStatus, get, listByGame, listByPlayer	host creates; join by invitation or, for public instances, by anyone; host and game sets status
plays	publish, saveVersion, get, list, link	participants of the instance publish; anyone reads
invitations	issue, redeem, revoke, status	participants issue; holder of the invitation secret redeems; issuer revokes
engagement	record, counts	any address, once per (address, play, kind) for kinds that count people

8.3 Instances

An instance is created by one deploy that news an unforgeable name for it. The client learns that name before deploying, from `POST /api/prepare-deploy` (Finding 3), which Embers's `instances/create/prepare` calls, so the Portal can show the instance, its link and its invitations before the deploy is finalised. The instance record is:

```
{ "id": <hex of the unforgeable name>, "game": "f1r3pix",
  "host": <address>, "createdAt": <ms>, "visibility": "public" | "unlisted" | "private",

  "status": "lobby" | "active" | "closed",
  "participants": { <address>: { "role": "host" | "player" | "reader",
                                "joinedAt": <ms> } },
  "gameEnv": <registry URI of the game's own environment>,
  "gameRef": <the game's handle for this instance's state> }
```

The game's own state for the instance lives in the game's environment, not in games. The create deploy calls the game's environment to initialise it (its `init` method, named in the manifest) and records the handle it returns. This fixes the mistake of global per-game state (Finding 14, item 5) at the point of creation: a game that cannot initialise a fresh instance cannot be registered.

8.4 Play records

A play record follows Embers's versioned-record shape: a header, small and listable, and a body, possibly large, kept under versions with a "latest" alias.

```
header: { "id", "game", "kind", "instance", "authors": [<address>],
          "title", "createdAt", "preview": <small, kind-specific>,
          "bodyHash": <blake2b-256 of the body>, "links": [<play id>] }
body:   <kind-specific: a pixel grid, a beat pattern, an ink set,
        a story's beats, a skein tune term, a performance trace>
```

list projects headers only. The kind is declared by the game's manifest: one kind for most games, two for F1R3Skein (tune, performance), linked in both directions through links (a tune names the performance it was snipped from; a performance names the tunes harvested from it). Bodies too large for the chain are stored by hash in F1R3Drive and only their hash recorded (Open decision 3).

8.5 Ordering and paging

Gallery views need recency and engagement order. The environment keeps, per game and kind, a time-bucketed index (`treeHashMap` from day number to the list of play ids published that day), and list pages by bucket. Engagement order is computed from counts. At small scale Embers serves both by exploratory deploys on an observer; at larger scale the gallery indexer serves them, rebuilding from block-finalised events and reads. Every page returned carries the block hash it reflects. Paging cost is Obligation 5.

8.6 Rendering and injection

Every template is rendered from typed values by `firefly-client` (Finding 8). Free text from players (titles, names, tags) is passed as `Value::String` and never spliced; byte payloads as `Value::Bytes`. Templates MUST contain no other interpolation.

9 Games

9.1 The game manifest

A game is a manifest registered in the games registry. The manifest is JSON; its hash is what the registry records, and the Portal fetches it from the game's origin and checks the hash.

```
{ "id": "f1r3pix", "name": "F1R3Pix", "tagline": "Collective_Visual_Intelligence",
  "entry": "https://pix.games.f1r3fly.io/", "platforms": ["web"],
  "env": "rho:id:...", // the game's own environment
  "init": { "template": "<hash>", "phloLimit": 200000 },
  "playTemplates": ["<hash>", "<hash>"], // signable under an allowance
  "otherTemplates": ["<hash>"], // always prompted
  "galleries": [ { "kind": "canvas", "renderer": "https://.../preview.html" } ],
  "contactsDialogue": true,
  "readerTier": true,
  "minProtocol": 1 }
```

The Portal's dashboard, galleries and launch flow are generated from the registry; adding a game is registering a manifest (Requirement 8).

9.2 The host protocol

A web game runs in a sandboxed frame on its own origin. It speaks to the shell by `postMessage`, with the shell checking the frame's origin against the manifest's entry. Version 1:

Message (game → shell)	Effect
<code>hello{protocol}</code>	Shell replies with the person's address, profile, sigil seed, and the instance context (id, role, participants, <code>gameRef</code>).
<code>deploy{template, values}</code>	The wallet renders, applies §7.2, prompts or uses the allowance, signs, and sends through Embers; replies with the deploy id, then its finalisation.
<code>read{template, values}</code>	An exploratory read through Embers at an observer; replies with the value and block hash.
<code>estimate{template, values}</code>	Cost quote only.
<code>publishPlay{kind, title, preview, body, links}</code>	Shell prepares plays/publish; same signing path.
<code>invite{}</code>	Opens the shared contact and invitation dialogue (§11) over the game; refused for games whose manifest sets <code>contactsDialogue: false</code> .
<code>engage{play, kind}</code>	Records engagement.
<code>subscribe{events}</code>	Forwards finalisation events relevant to the instance.
Message (shell → game)	
<code>context, event, locked</code>	Context changes, forwarded events, wallet locked (game should pause signing).

The game never sees a key and never constructs a deploy the wallet has not rendered itself.

9.3 Launch

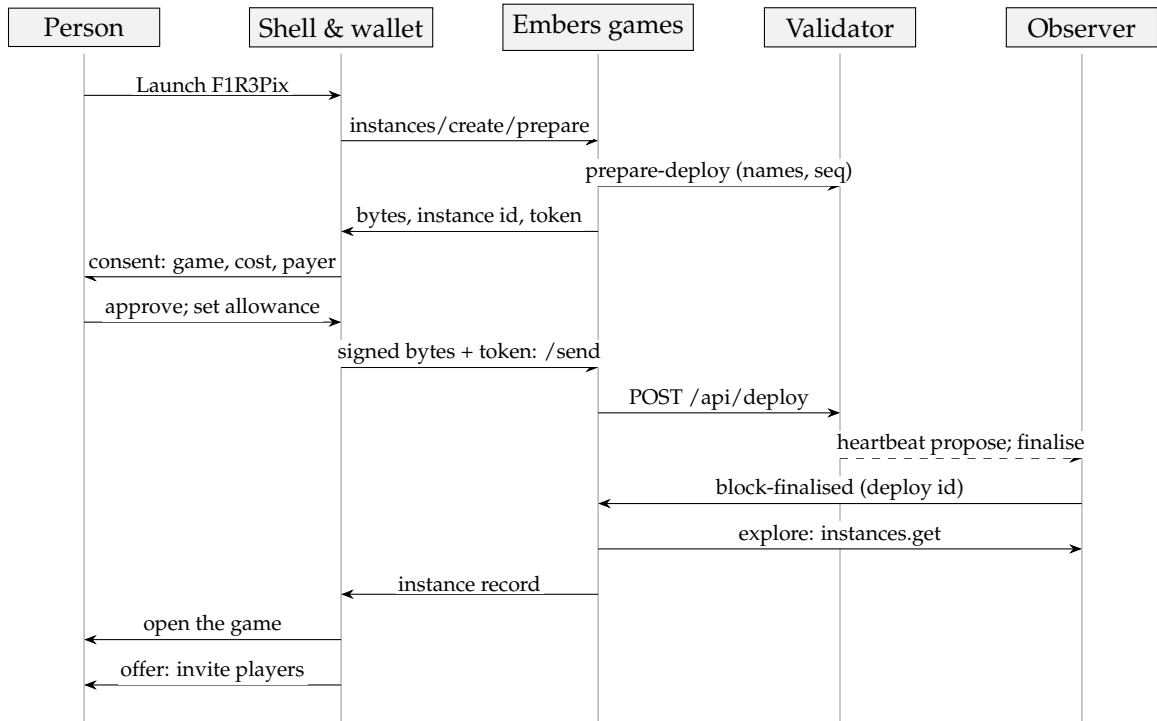


Figure 2: Launching an instance. The instance id is known at step 4, before the deploy, from prepare-deploy.

9.4 The games

Game	An instance is	A play record is	Integration
F1R3Pix	one shared canvas	the canvas at close, or a snapshot the players mark	web; contacts dialogue
F1R3Beat	one evolving pattern session	a pattern and its lineage	web; contacts dialogue; engagement feeds selection
F1R3Ink	one group colouring round	the set of inks given and received	web; contacts dialogue
F1R3SideChat	one story	arcs and beats; readers browse without a key (readerTier)	web; contacts dialogue

Game	An instance is	A play record is	Integration
F1R3Skein	one performance session	two kinds: performance (the gesture trace) and tune (a snapped term), cross-linked	Vision Pro instrument plus web browser; the browser sets <code>contactsDialogue: false</code> ; engagement feeds selection

Each game is rebuilt against this protocol and its own environment; the existing per-game Shard/ services are research, not a base (Finding 14).

9.5 F1R3Skein on Vision Pro

The instrument is not a web frame. It imports the person’s key file (the F1R3Sky format, by Air-Drop or file picker) into the device keychain, signs with the same deploy encoding, and calls the same Embers games endpoints, following the same signing rules. A Portal instance page shows a QR code and a universal link carrying the instance id, which the instrument opens to join the session and to publish performances and tunes into it.

10 Galleries

10.1 What the person sees

From the dashboard, each game card offers *Launch* and *Gallery*; a *Feed* tab interleaves all games. A gallery is an endlessly scrolling list of play headers, filterable by game, kind, participant and date, and sortable by recency, engagement, and (for kinds that have lineage) depth. Opening a play shows its preview rendered by the game’s sandboxed renderer, its participants, its links (for F1R3Skein, “hear this in context” from a tune and “tunes harvested here” from a performance) and its engagement counts, with *open in game* where the game supports it.

10.2 No sign-on to read

Galleries are read through Embers with no key: every read is an exploratory deploy on an observer, which costs no phlo. This also gives SideChat its free reader tier. A person who is not signed on sees an invitation to create a key at the point of first engagement, not before.

10.3 Trust in the view

Every page carries the block hash it was read at. A client MAY spot-check any header by its own exploratory read on an observer at that hash (explore-deploy-by-block-hash), so neither Embers nor the indexer can forge a gallery undetected.

11 Contacts and invitations

11.1 Sharing contacts with the Portal

The contact dialogue is the prototype's, rebuilt as one component used by every game whose manifest allows it.

Sources. A vCard or CSV file the person exports from her address book; the browser Contact Picker API where available (it returns only the entries the person selects); handles typed by hand for email, Discord, Slack, Mattermost, Telegram and others; and the addresses of players she has already played with, read from instance records.

Consent. Each import says what will be kept and where. Nothing is imported in the background, and nothing is read from an account by OAuth in this version.

Storage. Contacts are kept in IndexedDB, encrypted with AES-256-GCM under a key derived from the active private key by HKDF with a fixed, versioned context string, so the same contact book opens on any device that holds the key.

Backup. Optional: the encrypted contact book, padded to a size bucket, saved as a private record in the games environment, whose ciphertext only the key holder can read (Embers already stores encrypted payloads for agent teams). Open decision 4.

Deletion. Removing a contact removes it locally and from the next backup.

No contact's name, handle or email ever appears in a deploy, a template, a log or an Embers request. The Portal does not try to discover which contacts already play: hashed identifiers are guessable, and the invitation itself is the discovery.

11.2 The invitation

An invitation is a fresh secp256k1 key pair made by the inviter's wallet for that invitation: the *invite key*. Its public key is recorded on the shard; its private key travels to the invitee inside the link's fragment, which browsers do not send to servers:

```
https://games.f1r3fly.io/i/<instance id>#<invite private key, base64url>
```

invitations/issue records, under the invite public key: the instance, the issuer, the number of uses (one per invitee by default; more for an open link), an expiry block height, and an optional *stipend* from the issuer to each new player. To redeem, the invitee's wallet signs the invitee's own address with the invite key and deploys invitations/redeem with that signature; the contract checks it with `rho:crypto:secp256k1Verify`, so a copy of a pending redeem deploy cannot be re-used by another address, decrements the uses, adds the invitee to the instance, and pays the stipend. Appendix A sketches the contract.

This uses only master primitives. When `rho:system:capabilities` reaches master, issue can register a bounded-use capability keyed by the invite public key, and the link format stays the same.

11.3 Delivery

The Portal composes a message per invitee (game, instance, inviter's name, link) and delivers it through the person's own channels: the system share sheet (Web Share API), `mailto:`, copy to

clipboard, or a QR code. The operator sends nothing on the person’s behalf in this version. Relays through F1R3MTA or the F1R3FLY bot gateway are possible later; neither may put the recipient or the message on chain.

11.4 Redemption

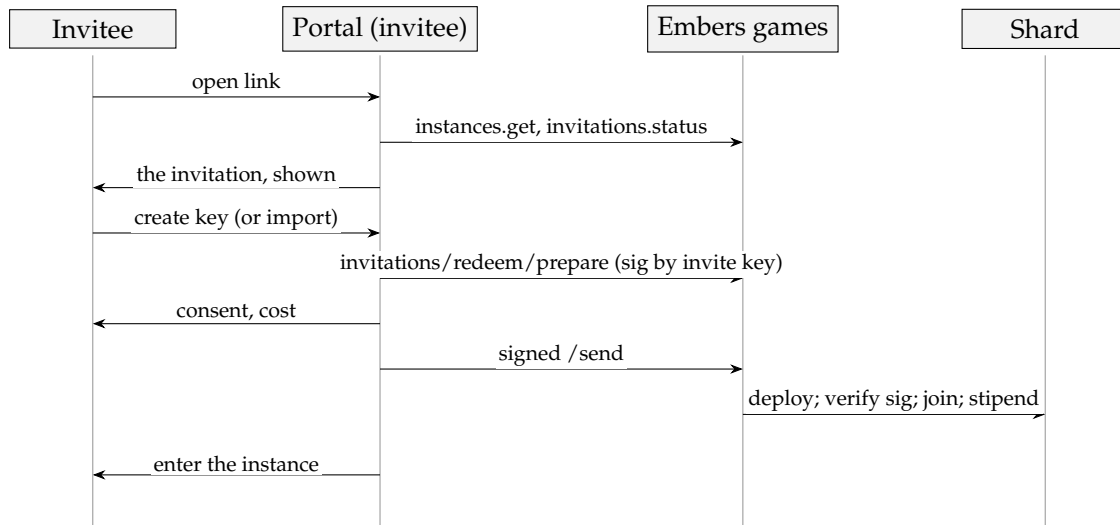


Figure 3: Redeeming an invitation. The invite key never leaves the invitee’s browser; only its signature over the invitee’s address is deployed.

11.5 Paying for a newcomer’s first deploy

A new key has an empty vault, and the redeem deploy costs phlo. Three routes, in order of preference:

1. **Testnets and local shards:** Embers’s testnet faucet (POST /testnet/wallet) funds the new address before redemption; ign1t10n’s genesis funds the local operator.
2. **The inviter pays first:** issue accepts a stipend escrow; at invitation time the Portal also offers to transfer a small amount directly to a *fresh* address it pre-generates for the invitee and places in the fragment alongside the invite key, so the invitee’s first key is already funded. (The invitee is told this key was made by the inviter and is offered to rotate to a new one, funded by transfer, at once.)
3. **After offered funding reaches master:** the redeem deploy is funded from the invitation’s private purse, and no pre-generated key is needed.

Which route production uses is Open decision 5.

12 Engagement

engagement/record counts plays, likes, opens, tips (as a link to a wallet transfer) and game-specific signals declared in the manifest (for F1R3Skein, *snipped-from* and *bred-from*). Kinds that

count people are limited to one per address per play. The Portal shows the counts; the games read them to drive selection. Weighting and resistance to gaming are the games' concern (§1), and the counts are public so that any weighting can be audited.

13 Security and privacy

Threat	Mitigation
Script injection in the shell	Strict Content Security Policy; no third-party script; the wallet module exposes only render-sign-send behind the rules of §7.2; keystore encrypted at rest; idle lock.
A malicious or buggy game	Sandboxed frame on its own origin; no key access; only registered templates; no deployer authority in game templates; allowances bound spend.
A dishonest Embers	Cannot sign; cannot alter a prepared deploy undetected (the wallet re-renders and compares); can at worst refuse service or waste a capped fee. Reads are spot-checkable by block hash.
A dishonest indexer	Same as reads from Embers.
Front-running a redemption	Redemption is a signature over the redeemer's address; copying it helps no other address.
Leaked invitation link	Bounded uses and expiry; issuer can revoke; the link grants entry to one instance and a stipend, nothing more.
Spam invitations	Issuing costs the issuer phlo; delivery is through the issuer's own channels.
Lost device	The key file the person downloaded at creation; no operator recovery. Multi-signature vaults (<code>rho:vault:multiSig</code>) are a later option.
Rholang injection	Typed rendering only (§8).
Linkability	One key links a person's plays across games; the wallet supports several keys and says so at creation.
Contact leakage	Contacts never leave the device unencrypted and never reach a deploy.

14 Deployment

Topology	Composition
Local	An <code>ign1t10n</code> shard (validators with heartbeat proposing, one observer), Embers with the games domain bound to it (<code>ign1t10n</code> can bundle Embers), and the Portal and games served from <code>localhost</code> . The testnet faucet funds new keys.
Testnet	An operator shard, Embers's testnet configuration, the Portal at a public origin, faucet enabled.

Topology	Composition
Production	Embers's mainnet configuration; faucet disabled; stipends from inviters.

Embers's configuration gains a `games_env_key`. The Portal's configuration names the Embers base URL, the shard id, the fee cap default, and the games origins it will frame.

15 The F1R3Gaze rendition

F1R3Gaze keys and F1R3Sky keys are the same files, so a person can carry one identity into F1R3Gaze today. A later phase publishes the Portal as a shard site (`f1r3://f1r3fly/games`) whose behaviour is `f1r3lang`, in which F1R3Gaze's wallet plays the part of the Portal's wallet and the host protocol maps onto capabilities the page holds. Nothing in the `games` environment changes.

16 Testing and acceptance

1. A key created in the Portal opens in F1R3Sky and F1R3Gaze and shows the same address; and conversely.
2. Two people on two browsers: one creates a key, launches an F1R3Pix instance and invites the other; the other redeems on a fresh browser with no key and plays; both see the same instance record and play record read from an observer.
3. A copied redeem deploy, re-signed by a third key, is rejected.
4. A game frame that asks to sign a term not in its manifest is refused without a prompt; one that exceeds its allowance gets a prompt.
5. A dishonest Embers that alters a prepared deploy's term or recipient is refused before signing.
6. Galleries load with no key; every page's block hash verifies against the observer.
7. F1R3Skein: a performance published from the instrument appears in the performance gallery with its tunes linked, and each tune links back.
8. The browser's network log shows no request to any server containing a contact.
9. No component in the Portal path calls `propose`.

17 Work packages and schedule

WP	Content	Effort
W0	Bring Embers's templates to master system names; pin F1R3Node-Rust, Embers and SDK versions; TypeScript port of <code>firefly-client</code> rendering with vectors.	2 dev-weeks
W1	Identity and wallet in the Portal: keystore, create/import/export, consent, signing rules, balance/history/transfer through Embers.	3 dev-weeks

WP	Content	Effort
W2	Embers games domain: environment, profiles, registry, instances (with prepare-deploy names), play records; regenerated SDK.	4 dev-weeks
W3	Portal shell: dashboard from the registry, launch, galleries, feed, spot-checking; host protocol v1.	3 dev-weeks
W4	Contacts and invitations: dialogue, encrypted store, invite keys, issue/redeem contract, stipends, faucet path.	3 dev-weeks
W5	First game ported: F1R3Pix (simplest state), end to end.	2 dev-weeks
W6	Remaining web games: Beat, Ink, SideChat (with reader tier).	5 dev-weeks
W7	F1R3Skein: browser galleries for tunes and performances; instrument key import and publishing.	3 dev-weeks
W8	Gallery indexer, engagement, security review, acceptance.	3 dev-weeks

W0–W5 (about 17 dev-weeks) yield a working Portal with one game; W6–W8 complete the suite. Allowances by private purse and the F1R3Gaze rendition follow when their dependencies land.

18 Obligations and decisions

18.1 Obligations

Obligation 1 (Name drift). Update Embers’s templates (`rho:rev:address` → `rho:vault:address` and the deploy-identity names) and confirm the `rho:deploy:data` tuple shape on the pinned F1R3Node-Rust revision; report the same drift to F1R3Gaze’s allow-list.

Obligation 2 (Rendering parity). Test vectors from `firefly-client` for every `Value` case, including strings with quotes, backslashes, newlines and non-ASCII, matched byte for byte by the TypeScript renderer.

Obligation 3 (SDK). Confirm how `@f1r3fly-io/embers-client-sdk` is published (F1R3Sky pins `~0.0.79`; it is not on the public npm registry) and that its signing matches the node’s `secp256k1` verification for `DeployData` including `shardId` and `expirationTimestamp`.

Obligation 4 (Proposing). Confirm heartbeat proposing on every target deployment and that Embers’s single bootstrap propose is acceptable there.

Obligation 5 (Paging). Measure `treeHashMap` bucket paging by exploratory deploy at 10^4 and 10^5 plays per game; decide when the indexer becomes mandatory.

Obligation 6 (Branch features). Track `rho:system:capabilities` and offered-funded deploys on their way to master; adopt each behind the existing interfaces when it arrives.

18.2 Open decisions

Open decision 1 (Home of the games domain). Inside the Embers repository as a peer of `wallets` and `agents`, or a crate Embers links. Proposed: inside, for one SDK and one deployment.

Open decision 2 (Unlocking the keystore). Passphrase only, or passkey (WebAuthn PRF) with passphrase fallback. Proposed: both.

Open decision 3 (Large bodies). Bodies above a threshold on chain anyway, or in F1R3Drive by hash. The F1R3MTA work took bodies on chain; consistency argues the same, cost argues otherwise.

Open decision 4 (Contact backup). Offer the encrypted on-shard backup in version 1, or keep contacts device-only.

Open decision 5 (Stipends). Default stipend amount, whether the pre-generated funded key of \$11.5 is offered in production, and whether the operator sponsors new players at all.

Open decision 6 (Registry governance). Who may register and retire games: the operator key now, CatchingF1R3 governance later?

Open decision 7 (Default visibility). Whether a new instance is public, unlisted or private by default, per game.

A Sketch of the invitation methods

Illustrative only; the implementation follows Embers's template layout and is rendered by firefly-client. `caller` is the address derivation of §8; instances and invites are tree-HashMaps of the environment; `currentBlock` reads `rho:block:data`.

```
contract games("@invitations", @issue, @invitePk, @instanceId,
               @uses, @expiresAt, @stipend, ret) = {
  new callerCh, okCh in {
    caller!(*callerCh) |
    for (@issuer <- callerCh) {
      isParticipant!(instanceId, issuer, *okCh) |
      for (@true <- okCh) {
        // stipend * uses is moved from the issuer's vault into the
        // environment's escrow vault (unforgeableAuthKey) in this deploy
        escrow!(issuer, stipend * uses, *okCh) |
        for (@true <- okCh) {
          treeHashMap!("set", invites, invitePk,
                      { "instance": instanceId, "issuer": issuer, "uses": uses,
                        "expiresAt": expiresAt, "stipend": stipend, "redeemed": Set() },
                      *ret)
        }
      }
    }
  }
}

contract games("@invitations", @redeem, @invitePk, @sig, ret) = {
  new callerCh, invCh, hashCh, verifyCh, blockCh in {
    caller!(*callerCh) | currentBlock!(*blockCh) |
    treeHashMap!("get", invites, invitePk, *invCh) |
    for (@guest <- callerCh & @inv <- invCh & @height <- blockCh) {
      if (inv == Nil or height > inv.get("expiresAt") or inv.get("uses") <= 0
          or inv.get("redeemed").contains(guest)) { ret!((false, "invalid")) }
    }
  }
}
```

```

else {
  blake2b256Hash!(guest.toByteArray(), *hashCh) |
  for (@digest <- hashCh) {
    secp256k1Verify!(digest, sig, invitePk, *verifyCh) |
    for (@true <- verifyCh) {
      treeHashMap!("set", invites, invitePk,
        inv.set("uses", inv.get("uses") - 1)
          .set("redeemed", inv.get("redeemed").add(guest)), *devNull) |
      join!(inv.get("instance"), guest, "player", *devNull) |
      payStipend!(guest, inv.get("stipend"), *devNull) |
      ret!((true, inv.get("instance")))
    }
  }
}
}
}
}
}
}

```

B Glossary

Allowance A budget within which the wallet signs a game’s registered play templates without prompting.

Environment A versioned contract inserted in the registry by Embers, holding a domain’s methods and state.

Instance One play of one game, created on the shard by its host.

Invite key A one-use key pair whose public half is recorded at issue and whose private half travels in the invitation link’s fragment.

Play record A gallery entry: a listable header and a versioned body.

Prepare / send Embers’s two-step write: the server renders, the client signs.